GAMEZONE

Plataforma de Torneos Gamer

Documento Técnico

Qué es cada parte del proyecto, para qué sirve y cómo se relaciona con el resto

Asignatura	Programación Back End (TI3V41) - Evaluación Sumativa N°3
Institución	INACAP Sede Curicó - Semestre Primavera 2026
Tecnologías	Python, Django 5.2, Django REST Framework 3.18, MySQL 8, Bootstrap 5.3
Repositorio	github.com/JoseFuenzalidaPino/gametest
Fecha	Octubre 2026

Contenido

1. Resumen del proyecto
2. Tecnologías utilizadas
3. Arquitectura (Modelo - Vista - Template)
4. Estructura de carpetas y archivos
5. Configuración (settings.py)
6. Modelos y base de datos
7. Formularios y validaciones
8. Vistas y rutas
9. Roles, sesiones y seguridad
10. Plantillas e interfaz
11. API REST
12. Flujos principales
13. Manejo de errores
14. Pruebas automatizadas
15. Instalación y ejecución
16. Cumplimiento de los criterios de evaluación
17. Limitaciones y trabajo pendiente

1. Resumen del proyecto

GAMEZONE es una aplicación web para gestionar torneos de videojuegos. Permite que los **jugadores** se registren, inicien sesión y se inscriban en torneos, y que los **administradores** creen, editen y eliminen torneos y sigan su avance por estados (Disponible, Completo, En curso y Finalizado).

El sistema está construido con **Django** siguiendo el patrón Modelo - Vista - Template, guarda la información en **MySQL**, usa **Bootstrap** para una interfaz responsiva y expone una **API REST** con Django REST Framework que responde en formato JSON.

Funcionalidades principales

- Registro de jugadores e inicio y cierre de sesión con roles (Jugador y Administrador).
- Página de inicio con un resumen: total de torneos, disponibles, en curso, finalizados, jugadores y próximos torneos.
- CRUD completo de torneos (crear, ver, editar, eliminar) restringido a administradores.
- Listado de torneos con filtros por videojuego, estado, rango de fechas y cantidad de participantes.
- Página de detalle de cada torneo con sus jugadores inscritos y sus resultados.
- Inscripción a torneos con control de cupos, de duplicados y de estado; el torneo pasa a "Completo" al llenarse.
- Panel del jugador (sus inscripciones) y panel del administrador (totales e inscritos por torneo).
- API REST protegida: lectura para usuarios autenticados, escritura solo para administradores.
- Páginas de error propias (403, 404 y 500) y 42 pruebas automatizadas.

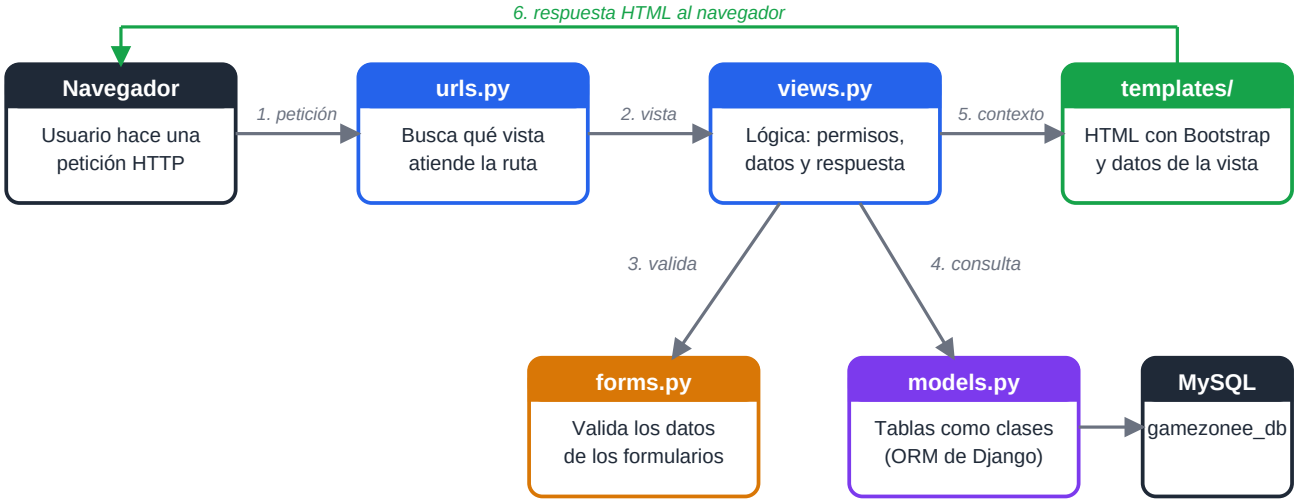
2. Tecnologías utilizadas

Las versiones exactas de las librerías están fijadas en **requirements.txt**, para que todos los equipos instalen lo mismo.

Tecnología	Versión	Para qué sirve en el proyecto
Python	3.11 o superior	Lenguaje en que está escrito todo el back end.
Django	5.2.18	Framework web: rutas, vistas, ORM, formularios, sesiones, autenticación, plantillas y panel /admin/.
Django REST Framework	3.18.3	Construye la API REST (/api/torneos/) con serialización JSON, autenticación y permisos.
MySQL	8.0	Base de datos relacional donde se guardan usuarios, jugadores, torneos, inscripciones y resultados.
mysqlclient / PyMySQL	2.2.8 / 1.2.3	Conectores que permiten a Django comunicarse con MySQL (Django usa mysqlclient).
Bootstrap	5.3.2 (CDN)	Estilos y componentes responsivos: barra de navegación, tarjetas, tablas, formularios y alertas.
asgiref, sqlparse, pytz, tzdata	según requirements	Dependencias internas de Django (servidor, formateo de SQL y zonas horarias).

3. Arquitectura (Modelo - Vista - Template)

Django organiza el código en tres capas. Cada petición del navegador recorre el siguiente camino:



Capa	Archivo(s)	Responsabilidad
Rutas	gamezone/urls.py, torneos/urls.py	Asocian cada URL (por ejemplo /torneos/5/) con la función de vista que la atiende.
Vista	torneos/views.py	Revisa permisos, lee o guarda datos usando modelos y formularios, y decide qué responder (HTML, redirección o error).
Formulario	torneos/forms.py	Valida lo que escribe el usuario antes de guardarlo y genera los mensajes de error.
Modelo	torneos/models.py	Define las tablas como clases de Python. Django traduce las consultas a SQL (ORM).
Template	torneos/templates/	HTML con etiquetas de Django que muestra los datos que entrega la vista.
API	serializers.py, permissions.py, ViewSet	Misma lógica de datos, pero responde JSON en vez de HTML.

4. Estructura de carpetas y archivos

El proyecto Django se llama **gamezone** (configuración general) y contiene una aplicación llamada **torneos** (toda la funcionalidad).

```
gametest/
|-- manage.py                  Comando principal de Django (runserver, migrate, test...)
|-- requirements.txt           Librerías y versiones a instalar
|-- run_lan.ps1                Script para levantar el servidor en la red local
|-- .gitignore                 Archivos que no se suben a Git (.venv, __pycache__)
|-- gamezone/                  CONFIGURACIÓN DEL PROYECTO
|   |-- settings.py            Base de datos, apps, seguridad, idioma, API
|   |-- urls.py                Rutas principales del sitio
|   |-- wsgi.py / asgi.py      Puntos de entrada para servidores de producción
|-- torneos/                   APLICACIÓN PRINCIPAL
    |-- models.py              Tablas: Jugador, Torneo, Inscripcion, Resultado
    |-- forms.py               Formularios y validaciones
    |-- views.py               Lógica de cada página y de la API
    |-- urls.py                Rutas de /torneos/...
    |-- roles.py               Quién es administrador y decorador admin_requerido
    |-- permissions.py         Permiso de la API (lectura / escritura)
    |-- serializers.py         Conversión Torneo <-> JSON para la API
    |-- context_processors.py  Variable es_admin disponible en todas las plantillas
    |-- admin.py               Configuración del panel /admin/ de Django
    |-- apps.py                Registro de la aplicación
    |-- tests.py               42 pruebas automatizadas
    |-- migrations/            Historial de cambios de la base de datos (0001 a 0005)
    |-- management/commands/
    |   |-- runserver.py       runserver escucha en 0.0.0.0:8000 (red local)
    |-- templates/             Páginas HTML (ver sección 10)
```

Archivo	Para qué sirve
manage.py	Herramienta de línea de comandos de Django. Se usa para levantar el servidor, aplicar migraciones, crear usuarios administradores y correr las pruebas.
gamezone/settings.py	Configuración central: conexión a MySQL, aplicaciones instaladas, seguridad, idioma, validadores de contraseña y configuración de la API.
gamezone/urls.py	Rutas raíz: inicio/login, registro, logout, paneles, /admin/, /torneos/ y /api/.
torneos/models.py	Define la estructura de los datos. Cada clase es una tabla en MySQL.
torneos/forms.py	Formularios de torneo, filtros, login y registro, con sus reglas de validación.
torneos/views.py	Una función por página: qué datos busca, quién puede entrar y qué responde.
torneos/roles.py	Centraliza la regla "es administrador" y el decorador que protege las vistas de administración.
torneos/permissions.py	Regla de acceso de la API: cualquier usuario autenticado lee, solo administradores escriben.
torneos/serializers.py	Transforma objetos Torneo a JSON y valida los datos que llegan por la API.
torneos/context_processors.py	Agrega la variable *es_admin* a todas las plantillas para mostrar u ocultar opciones del menú.
torneos/admin.py	Registra los 4 modelos en el panel /admin/ con columnas, búsqueda y filtros.
torneos/tests.py	Pruebas que verifican automáticamente login, registro, permisos, CRUD, validaciones, inscripciones y API.

Archivo	Para qué sirve
torneos/migrations/	Archivos generados por Django que crean y modifican las tablas en MySQL de forma ordenada.
run_lan.ps1	Levanta el servidor accesible desde otros equipos de la red y muestra las direcciones para entrar.

5. Configuración (settings.py)

Las opciones sensibles o que cambian entre equipos se leen desde **variables de entorno**, con un valor por defecto. Así el mismo código funciona en cualquier computador sin modificar archivos.

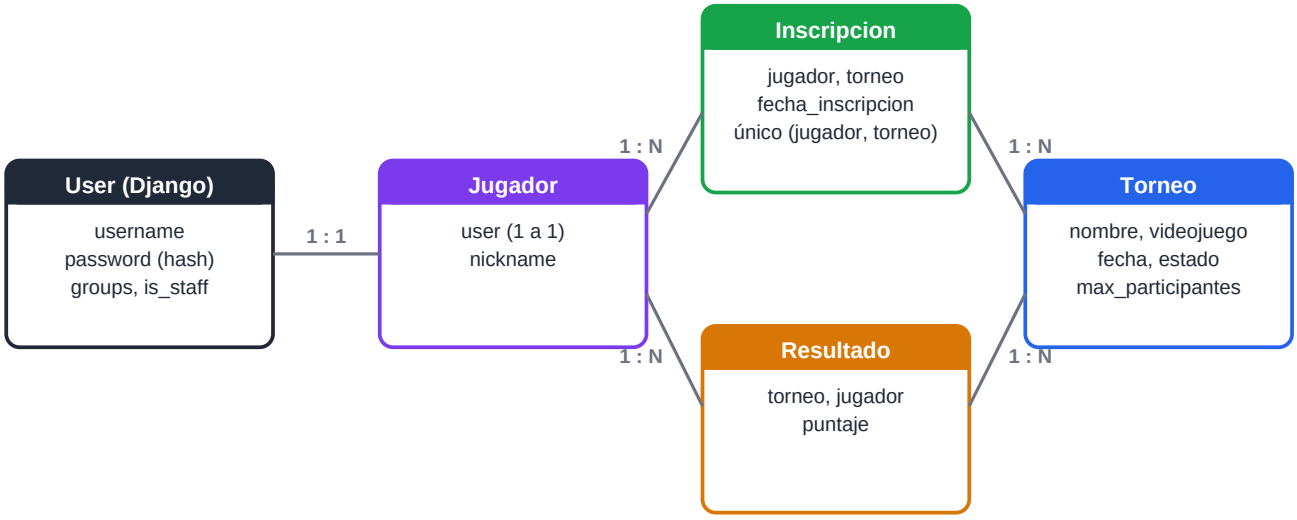
Configuración	Valor / origen	Para qué sirve
DATABASES	MySQL, base gamezonee_db, utf8mb4	Conexión a la base de datos. utf8mb4 permite guardar tildes, ñ y emojis.
INSTALLED_APPS	admin, auth, sessions, messages, torneos, staticfiles, rest_framework	Módulos activos. *torneos* va antes de *staticfiles* para que su comando runserver tenga prioridad.
MIDDLEWARE	Session, Csrf, Authentication, Messages, Clickjacking...	Capas que procesan cada petición: sesiones, protección CSRF, usuario actual, mensajes y protección contra iframes.
TEMPLATES	APP_DIRS = True + context processor *roles*	Busca plantillas dentro de torneos/templates y agrega *es_admin* a todas.
AUTH_PASSWORD_VALIDATORS	4 validadores de Django	Contraseñas de al menos 8 caracteres, no comunes, no solo números y no parecidas al usuario.
LANGUAGE_CODE	es-es	Mensajes de Django (errores de formularios, etc.) en español.
LOGIN_URL	'login' (la ruta /)	A dónde se envía a quien intenta entrar a una página protegida sin sesión.
DEBUG	DJANGO_DEBUG (por defecto 1)	Con 1 muestra errores técnicos detallados; con 0 muestra las páginas de error propias.
ALLOWED_HOSTS	localhost, 127.0.0.1, nombre e IPs del equipo + DJANGO_ALLOWED_HOSTS	Direcciones a las que el servidor acepta responder. Se detectan solas para permitir el acceso desde la red local.
SECRET_KEY	DJANGO_SECRET_KEY	Clave para firmar sesiones y tokens CSRF. En producción debe definirse por variable de entorno.
REST_FRAMEWORK	Session + Basic auth; IsAuthenticated	La API exige iniciar sesión, ya sea con la sesión del sitio o con usuario y contraseña.

Variables de entorno disponibles

Variable	Valor por defecto	Uso
DB_NAME	gamezonee_db	Nombre de la base de datos MySQL.
DB_USER / DB_PASSWORD	root / 1234	Usuario y contraseña de MySQL.
DB_HOST / DB_PORT	localhost / 3306	Servidor y puerto de MySQL.
DJANGO_DEBUG	1	Poner 0 para ver las páginas 403/404/500 propias.
DJANGO_SECRET_KEY	clave de desarrollo	Clave secreta propia para producción.
DJANGO_ALLOWED_HOSTS	(vacío)	Direcciones extra separadas por coma, por ejemplo 192.168.1.88.

6. Modelos y base de datos

Los modelos están en **torneos/models.py**. Para los usuarios se reutiliza el modelo **User** de Django, que ya maneja contraseñas cifradas, grupos y permisos.



Torneo

Campo	Tipo	Descripción
nombre	Texto (100)	Nombre del torneo. El formulario exige que sea único y tenga al menos 3 caracteres.
videojuego	Texto (100)	Juego del torneo; se usa en los filtros.
fecha	Fecha	Día del torneo. No puede ser pasada al crearlo o al cambiarla.
max_participantes	Entero (2 a 1000)	Cupos del torneo. Validado en el modelo, el formulario y la API.
descripcion	Texto largo	Información del torneo (mínimo 10 caracteres).
estado	Opción	DISPONIBLE, COMPLETO, EN_CURSO o FINALIZADO. Por defecto DISPONIBLE.
cupos_disponibles	Entero	Campo heredado de una versión anterior; ya no se usa (los cupos se calculan con las inscripciones).

Métodos del modelo: **inscritos_count()** cuenta las inscripciones; **admite_inscripciones()** indica si el estado es Disponible; **actualizar_estado_por_cupos()** cambia el estado a Completo cuando se llena.

Jugador, Inscripcion y Resultado

Modelo	Campos	Para qué sirve
Jugador	user (1 a 1 con User), nickname	Perfil de jugador asociado a una cuenta. Se crea al registrarse o en la primera inscripción.
Inscripcion	jugador, torneo, fecha_inscripcion	Relaciona un jugador con un torneo. **unique_together** impide inscribirse dos veces en el mismo torneo, incluso a nivel de base de datos.
Resultado	torneo, jugador, puntaje	Puntaje obtenido por un jugador en un torneo. Se muestra ordenado en el detalle del torneo.

Al eliminar un torneo o un jugador, sus inscripciones y resultados se eliminan en cascada (*on_delete=CASCADE*), para no dejar datos huérfanos.

Migraciones

Migración	Qué hace
0001_initial	Crea las tablas Jugador, Torneo, Inscripcion y Resultado.
0002_torneo_cupos_disponibles	Agrega el campo cupos_disponibles.
0003_alter_torneo_estado	Define las cuatro opciones de estado del torneo.
0004_crear_grupos	Crea los grupos (roles) **Usuarios** y **Administrador**.
0005_validar_max_participantes	Agrega el rango permitido 2 a 1000 para max_participantes.

7. Formularios y validaciones

Todos los formularios están en **torneos/forms.py**. Django valida los datos en el servidor, así que las reglas se cumplen aunque alguien salte las validaciones del navegador.

Formulario	Usado en	Para qué sirve
BootstrapMixin	Todos	Agrega las clases de Bootstrap a cada campo y marca en rojo (is-invalid) los que tienen errores.
TorneoForm	Crear y editar torneo	Formulario del torneo con selector de fecha y todas las validaciones de negocio.
TorneoFiltroForm	Listado de torneos	Valida los filtros de búsqueda y los aplica a la consulta.
LoginForm	Inicio (/)	Basado en AuthenticationForm de Django: comprueba usuario y contraseña.
RegistroForm	Registro (/register/)	Basado en UserCreationForm: usuario único, contraseña confirmada y segura.

Reglas de validación de TorneoForm

Método	Regla	Mensaje de error
clean_nombre	Mínimo 3 caracteres y no repetido (sin importar mayúsculas)	"Ya existe un torneo con ese nombre."
clean_videojuego	Obligatorio, sin espacios sobrantes	"Debes indicar el videojuego."
clean_fecha	No puede ser anterior a hoy al crear o al cambiar la fecha (sí se puede editar un torneo pasado sin tocar su fecha)	"La fecha no puede ser anterior a hoy."
validadores del modelo	max_participantes entre 2 y 1000	Mensaje estándar de Django en español
clean_descripcion	Mínimo 10 caracteres	"La descripción debe tener al menos 10 caracteres."
clean	Al editar: el máximo no puede quedar bajo los inscritos, y un torneo lleno no puede quedar como Disponible	"No puede ser menor a los N jugadores ya inscritos."

8. Vistas y rutas

Cada ruta se define en **urls.py** y apunta a una función en **views.py**. La columna "Acceso" indica quién puede usarla.

Ruta	Vista	Acceso	Qué hace
/	iniciar_sesion	Público	Página de inicio: resumen de torneos y próximos torneos. Sin sesión muestra el login; al entrar redirige al panel según el rol.
/register/	registrar_usuario	Público	Crea la cuenta, la agrega al grupo Usuarios y crea su Jugador.
/logout/	cerrar_sesion	Con sesión	Cierra la sesión y vuelve al inicio.
/panel/	jugador_panel	Con sesión	Lista las inscripciones del usuario.
/panel/admin/	admin_panel	Administrador	Totales de torneos, jugadores e inscripciones e inscritos por torneo.
/torneos/	lista_torneos	Público	Listado con filtros, inscritos por torneo y estado.
/torneos/<id>/	detalle_torneo	Público	Datos del torneo, cupos libres, jugadores inscritos, resultados y botón de inscripción.
/torneos/crear/	crear_torneo	Administrador	Formulario para crear un torneo.
/torneos/editar/<id>/	editar_torneo	Administrador	Formulario para modificar un torneo (incluido su estado).
/torneos/eliminar/<id>/	eliminar_torneo	Administrador	GET muestra la confirmación; POST elimina.
/torneos/inscribirse/<id>/	inscribirse_torneo	Con sesión (POST)	Inscribe al jugador validando duplicado, estado y cupos.
/api/torneos/	TorneoViewSet	Autenticado	API REST de torneos (ver sección 11).
/admin/	admin.site	Staff	Panel de administración de Django para los 4 modelos.

Funciones de apoyo en views.py

- **obtener_jugador(user)**: devuelve el Jugador de un usuario o lo crea si no existe.
- **torneos_con_inscritos()**: consulta de torneos que agrega la cantidad de inscritos (Count) en una sola consulta SQL.
- **redirigir_a_panel(user)**: envía al administrador a /panel/admin/ y al jugador a /panel/.

9. Roles, sesiones y seguridad

Roles

Los roles se implementan con los **grupos** de Django, creados por la migración 0004:

Rol	Cómo se obtiene	Qué puede hacer
Visitante	Sin iniciar sesión	Ver el inicio, el listado y el detalle de torneos; registrarse.
Jugador	Al registrarse queda en el grupo **Usuarios**	Todo lo anterior, más inscribirse en torneos, ver su panel y leer la API.
Administrador	Grupo **Administrador** o usuario staff/superusuario	Todo lo anterior, más crear, editar y eliminar torneos, cambiar estados, ver el panel de administración y escribir en la API.

La regla está en **torneos/roles.py**:

```
def es_administrador(user):
    return user.is_staff or user.groups.filter(name="Administrador").exists()
def admin_requerido(vista):          # decorador para vistas de administración
    @wraps(vista)
    @login_required                  # sin sesión -> redirige al login
    def envoltura(request, *args, **kwargs):
        if not es_administrador(request.user):
            raise PermissionDenied   # jugador -> error 403
        return vista(request, *args, **kwargs)
    return envoltura
```

Sesiones

Django guarda la sesión en la base de datos (tabla django_session) y entrega al navegador una cookie con un identificador. Al iniciar sesión, **login()** asocia el usuario a esa sesión; al cerrar, **logout()** la borra. Mientras la sesión existe, cada página conoce al usuario actual mediante **request.user**.

Medidas de seguridad implementadas

Medida	Dónde	Contra qué protege
Contraseñas cifradas (hash PBKDF2)	Modelo User de Django	Nunca se guarda la contraseña en texto plano.
Validadores de contraseña	settings.py / RegistroForm	Contraseñas débiles o fáciles de adivinar.
Token CSRF en todos los formularios	{% csrf_token %} + CsrfViewMiddleware	Que otro sitio envíe formularios en nombre del usuario.
Cambios solo por POST	Inscripción y eliminación	Que un enlace o un buscador ejecute acciones por accidente.
Control de roles	admin_requerido, EsAdministradorOSoloLectura	Que un jugador o visitante modifique torneos.
Validación del parámetro next	iniciar_sesion	Redirecciones a sitios externos maliciosos después del login.
Inscripción única en la base de datos	unique_together + IntegrityError	Inscripciones duplicadas por doble clic.
ALLOWED_HOSTS	settings.py	Peticiones con encabezados Host falsos.
Credenciales por variables de entorno	settings.py	Publicar contraseñas de la base de datos en el código.
Botón para mostrar la contraseña	base.html	Errores de tipeo; se vuelve a ocultar antes de enviar el formulario.

10. Plantillas e interfaz

Todas las páginas heredan de **base.html** con *{% extends %}* y rellenan el bloque *contenido*. Las piezas que se repiten se reutilizan con *{% include %}*.

Plantilla	Para qué sirve
torneos/base.html	Estructura común: meta viewport, Bootstrap, barra de navegación que cambia según la sesión y el rol, mensajes, pie de página y el script del ojo para mostrar contraseñas.
torneos/inicio.html	Inicio: tarjetas de resumen, próximos torneos y formulario de login si no hay sesión.
torneos/lista.html	Formulario de filtros y tabla responsiva de torneos.
torneos/detalle.html	Detalle del torneo con inscritos, resultados y botón de inscripción.
torneos/crear.html / editar.html	Páginas de creación y edición; ambas usan el mismo formulario parcial.
torneos/_form_torneo.html	Formulario de torneo reutilizable: campos, errores por campo y botones.
torneos/eliminar.html	Confirmación antes de eliminar; avisa si el torneo tiene inscripciones.
torneos/_estado_badge.html	Etiqueta de color del estado (verde, amarillo, azul o gris).
usuarios/registro.html	Formulario de registro de jugadores.
usuarios/jugador_panel.html	Tabla con las inscripciones del jugador.
usuarios/admin_panel.html	Totales y tabla de torneos con acciones de administración.
403.html / 404.html / 500.html	Páginas de error propias (se ven con DJANGO_DEBUG=0).

Diseño responsivo

- **meta viewport**: el sitio se adapta al ancho del celular en lugar de verse como escritorio reducido.
- **Grilla de Bootstrap** (row / col-md / col-lg): las tarjetas y columnas se reordenan según el tamaño de pantalla.
- **navbar-expand-lg**: en pantallas chicas el menú se pliega en un botón.
- **table-responsive**: las tablas se desplazan horizontalmente en vez de romper el diseño.

11. API REST

La API se construye con Django REST Framework. **TorneoViewSet** (un ModelViewSet) genera automáticamente todas las operaciones CRUD y el **router** de gamezone/urls.py crea las rutas.

Método	Ruta	Acción	Quién
GET	/api/torneos/	Lista todos los torneos	Usuario autenticado
POST	/api/torneos/	Crea un torneo	Administrador
GET	/api/torneos/<id>/	Obtiene un torneo	Usuario autenticado
PUT / PATCH	/api/torneos/<id>/	Modifica un torneo (completo o parcial)	Administrador
DELETE	/api/torneos/<id>/	Elimina un torneo	Administrador

Pieza	Para qué sirve
serializers.py (TorneoSerializer)	Convierte el torneo a JSON, agrega el campo calculado *inscritos* y valida fecha y máximo igual que el formulario web.
permissions.py (EsAdministradorOSoloLectura)	Sin autenticar responde 401/403; un jugador solo puede usar GET; un administrador puede usar todos los métodos.
REST_FRAMEWORK en settings.py	Habilita autenticación por sesión (desde el navegador) y Basic (usuario y contraseña, por ejemplo con curl o Postman).

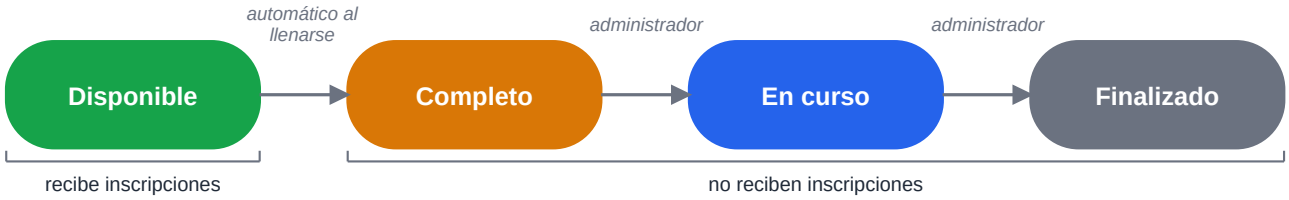
Ejemplo de uso (con usuario y contraseña)

```
curl -u usuario:clave http://localhost:8000/api/torneos/
[
  {
    "id": 1,
    "nombre": "Copa Valorant",
    "videojuego": "Valorant",
    "fecha": "2026-11-20",
    "max_participantes": 16,
    "descripcion": "Torneo 5 vs 5 eliminación directa",
    "estado": "DISPONIBLE",
    "inscritos": 3
  }
]
```

Si los datos no son válidos, la API responde **400** con el detalle de cada error, por ejemplo *{"fecha": ["La fecha no puede ser anterior a hoy."]}*.

12. Flujos principales

Estados del torneo



Solo los torneos **Disponibles** aceptan inscripciones. Cuando se inscribe el último jugador, el sistema cambia el estado a **Completo** automáticamente. El administrador pasa el torneo a **En curso** y a **Finalizado** desde el formulario de edición.

Registro e inicio de sesión

1. El visitante entra a /register/ y completa usuario, contraseña y confirmación.
2. RegistroForm valida que el usuario no exista y que la contraseña sea segura.
3. Se crea el User, se agrega al grupo Usuarios y se crea su Jugador.
4. Se redirige a / con el mensaje "Cuenta creada, ya puedes iniciar sesión".
5. Al iniciar sesión, el jugador va a /panel/ y el administrador a /panel/admin/.

Inscripción a un torneo

1. El jugador abre el detalle del torneo y presiona **Inscribirse** (formulario POST con token CSRF).
2. Si no hay sesión, se le envía al login y luego vuelve al torneo.
3. La vista revisa en orden: que no esté ya inscrito, que el torneo esté Disponible y que queden cupos.
4. Si todo está bien, crea la Inscripcion y, si era el último cupo, marca el torneo como Completo.
5. Vuelve al detalle con un mensaje de éxito o de error.

Gestión de torneos (administrador)

- Crear: /torneos/crear/, el formulario valida los datos y al guardar muestra el detalle del torneo.
- Editar: permite cambiar fecha, descripción, máximo de jugadores y estado.
- Eliminar: pide confirmación y elimina también sus inscripciones y resultados.

13. Manejo de errores

Situación	Cómo se maneja	Respuesta
Torneo que no existe	get_object_or_404	Página 404 "El torneo o la página que buscas no existe".
Jugador intenta administrar	admin_requerido lanza PermissionDenied	Página 403 "No tienes permisos...".
Visitante entra a una página protegida	login_required	Redirección al login con ?next= para volver después.
Formulario de torneo inválido	TorneoForm	Se vuelve a mostrar con los campos en rojo y su mensaje.
Inscripción duplicada	Consulta previa + IntegrityError	Mensaje "Ya estás inscrito en este torneo".
Torneo no disponible	admite_inscripciones()	Mensaje con el estado actual del torneo.
Sin cupos	Conteo de inscripciones	Mensaje "No hay cupos disponibles" y el torneo pasa a Completo.
Inscripción por GET	require_POST	Error 405 (método no permitido).
Datos inválidos en la API	TorneoSerializer	Error 400 con el detalle en JSON.
Error inesperado del servidor	Plantilla 500.html	Página de error genérica (sin mostrar detalles técnicos).

14. Pruebas automatizadas

Las pruebas de **torneos/tests.py** crean una base de datos temporal, ejecutan cada caso y la eliminan. Se corren con **python manage.py test torneos**. Resultado actual: **42 pruebas, todas correctas**.

Grupo de pruebas	N°	Qué verifica
InicioTests	5	Rutas / y /register/, resumen de torneos en el inicio, login visible solo sin sesión, meta viewport.
RegistroTests	5	Creación de usuario, grupo y jugador; usuario duplicado; contraseñas distintas o débiles.
LoginTests	5	Redirección según rol, parámetro next seguro, credenciales incorrectas y logout.
ListaYDetalleTests	4	Datos del listado, todos los filtros, inscritos y resultados en el detalle, 404.
PermisosCrudTests	5	Visitante va al login, jugador recibe 403, administrador accede, páginas de error propias.
CrudTorneoTests	6	Crear válido e inválido, nombre duplicado, máximo bajo inscritos, editar torneo pasado, eliminar con confirmación.
InscripcionTests	8	Login y POST obligatorios, duplicado, paso a Completo, sin cupos, torneo no disponible, 404, panel del jugador.
ApiTests	4	Acceso anónimo bloqueado, jugador solo lectura, administrador crea y recibe errores de validación, Basic auth.

15. Instalación y ejecución

1. Preparar el entorno

```
git clone https://github.com/JoseFuenzalidaPino/gametest.git
cd gametest
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

2. Crear la base de datos en MySQL

```
CREATE DATABASE gamezonee_db CHARACTER SET utf8mb4;
```

3. Configurar la conexión (solo si MySQL no usa root / 1234)

```
$env:DB_PASSWORD = "mi_clave"      # PowerShell; también DB_USER, DB_HOST, DB_PORT, DB_NAME
```

4. Crear las tablas y un administrador

```
python manage.py migrate              # crea tablas y los grupos Usuarios / Administrador
python manage.py createsuperuser      # usuario administrador
```

5. Levantar el servidor

```
python manage.py runserver            # escucha en 0.0.0.0:8000
# o bien:
powershell -ExecutionPolicy Bypass -File .\run_lan.ps1
```

Desde el mismo equipo se entra a **http://localhost:8000/**. Desde otro equipo de la misma red se usa la IP del servidor, por ejemplo **http://192.168.1.88:8000/**.

> **Acceso desde la red:** el puerto 8000 debe estar permitido en el firewall de Windows del equipo que ejecuta el servidor, para el tipo de red en uso (Privada o Pública). Si aparece "Bad Request (400)", se agrega la IP con *$env:DJANGO_ALLOWED_HOSTS="192.168.1.88"* antes de levantar el servidor.

Comandos útiles

Comando	Para qué sirve
python manage.py runserver	Levanta el servidor de desarrollo.
python manage.py migrate	Aplica las migraciones pendientes a la base de datos.
python manage.py makemigrations	Genera migraciones nuevas después de cambiar models.py.
python manage.py createsuperuser	Crea un usuario administrador.
python manage.py test torneos	Ejecuta las 42 pruebas automatizadas.
python manage.py check	Revisa que la configuración no tenga errores.

16. Cumplimiento de los criterios de evaluación

#	Criterio	Dónde se cumple en el proyecto
1	Proyecto Django creado y configurado	Proyecto *gamezone* con la app *torneos*; settings.py configurado (apps, middleware, idioma, seguridad, API). *manage.py check* sin errores.
2	MySQL configurado y conectado	DATABASES con motor MySQL, utf8mb4 y credenciales por variables de entorno; 5 migraciones aplicadas.
3	CRUD de torneos	Vistas lista, detalle, crear, editar y eliminar (web) y TorneoViewSet (API); protegidos por rol.
4	Formulario de creación	crear.html + _form_torneo.html + TorneoForm con estilos Bootstrap y selector de fecha.
5	Listado con detalles	lista.html con filtros, inscritos y estado; detalle.html con descripción, cupos, inscritos y resultados.
6	Plantilla base responsiva	base.html con Bootstrap 5.3, meta viewport, menú plegable, grilla y tablas responsivas.
7	Página de inicio con resumen	inicio.html: totales por estado, cantidad de jugadores y próximos torneos.
8	Rutas en urls.py	gamezone/urls.py y torneos/urls.py con nombres de ruta usados en las plantillas ({% url %}).
9	Validación de formularios	Métodos clean_* y clean de TorneoForm; validadores del modelo; misma validación en la API.
10	Manejo de errores	404/403/500 propios, mensajes de inscripción, require_POST, IntegrityError y errores 400 en la API.
11	Presentación	Este documento, el sitio en funcionamiento y las pruebas automatizadas como respaldo.

17. Limitaciones y trabajo pendiente

Aspectos de la pauta que hoy están cubiertos solo en parte y que se podrían mejorar:

Tema	Estado actual	Mejora propuesta
Resultados	Se ven en el detalle del torneo, pero se registran solo desde /admin/.	Formulario web para que el administrador cargue puntajes.
Informes y métricas	Totales por estado, jugadores e inscritos por torneo.	Ranking de torneos con más participación y de jugadores con mejores resultados.
API	Solo cubre torneos.	Endpoints para jugadores, inscripciones y resultados.
Campo cupos_disponibles	Ya no se usa.	Eliminarlo con una migración.
Producción	Servidor de desarrollo (runserver) y SECRET_KEY de desarrollo por defecto.	Servidor WSGI (gunicorn/waitress), DEBUG=0 y SECRET_KEY propia.